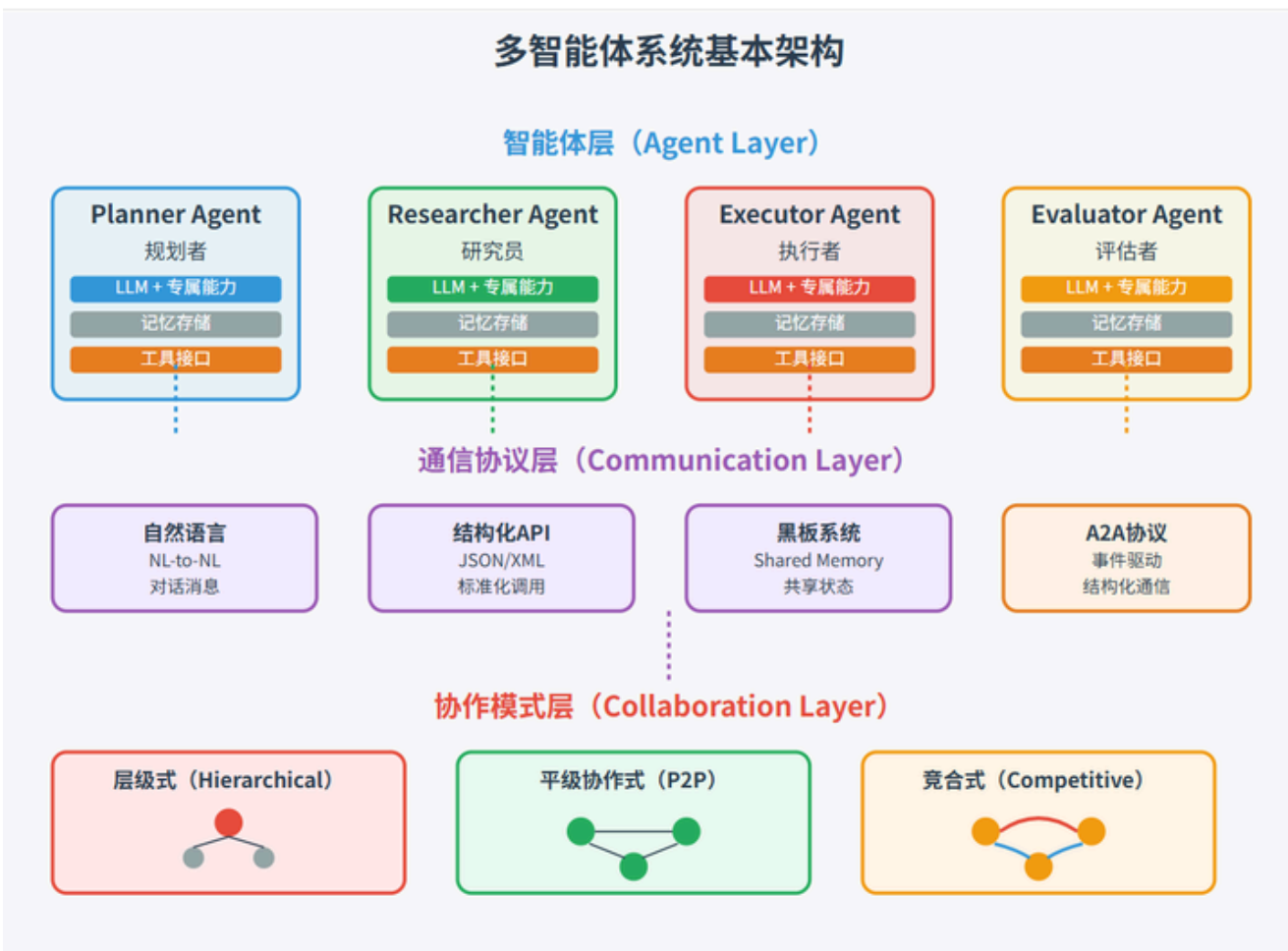


实验五：基于多智能体协同的前后端系统构建实验说明

1. 多智能体系统简介

多智能体系统是指由多个具备不同职责的智能体共同完成复杂任务的系统形态。与单一模型直接输出结果不同，多智能体系统强调分工协作：有的智能体负责规划，有的负责实现，有的负责界面设计，还有的负责评审和反馈。通过这种协同机制，系统能够更清晰地完成需求分析、任务拆分、功能实现和结果验证。

本实验中的多智能体系统包括规划智能体、后端智能体、前端智能体和视觉评审智能体四个角色，它们共同参与前后端系统构建过程。



2. 实验目标

本实验的核心目标是通过多智能体编程，真实实现一个前后端系统。实验过程中，学生需要组织规划、后端、前端和视觉评审四类智能体协同工作，使其围绕同一任务完成系统设计、接口组织、界面生成与结果评审，并最终形成一个可运行、可交互、可观察的系统原型。

完成本实验后，应达到以下目标：

1. 理解多智能体协同开发的完整流程。
2. 能够观察需求分析、后端设计、前端设计、视觉评审之间的上下游关系。
3. 能够运行一个真实的前后端系统，而不是只看到文字设计结果。
4. 能够独立组织一次多智能体编程过程，并对生成结果进行验证与分析。

3. 实验思路

本实验采用多智能体协同编程流程：

1. 规划智能体给出系统摘要、目标、架构、API 和任务分解。
2. 后端智能体基于规划结果输出服务名称、接口、数据模型和测试点。
3. 前端智能体结合规划结果和后端结果输出页面分区、交互流程和 UI 建议。
4. 视觉评审智能体结合结构图与前三个智能体结果，对系统架构可视化和界面组织方式进行分析。
5. 前端页面根据这些真实结果动态生成原型，并提供交互动效。

因此，本实验的重点不在于展示一个固定页面，而在于完整展示多智能体编程如何推动前后端系统从方案生成走向可视化原型实现。

4. 技术方案

4.1 技术栈

本实验采用如下技术栈：

1. 后端：Node.js + Express
2. 模型接入：@openrouter/sdk 与 OpenRouter 兼容接口
3. 前端：HTML + CSS + JavaScript
4. 数据交换格式：JSON

4.2 智能体分工

本实验使用四个主智能体角色：

1. `inclusionai/ling-2.6-1t:free` - 角色：规划智能体 - 任务：输出 `summary / goals / architecture / api / tasks`
2. `qwen/qwen3-coder:free` - 角色：后端智能体 - 任务：输出 `service_name / endpoints / data_models / test_points`
3. `openai/gpt-oss-120b:free` - 角色：前端智能体 - 任务：输出 `page_title / sections / interactions / ui_notes`
4. `meta-llama/llama-3.2-3b-instruct:free` - 角色：视觉评审智能体 - 任务：输出 `screen_summary / visible_modules / ui_observations`

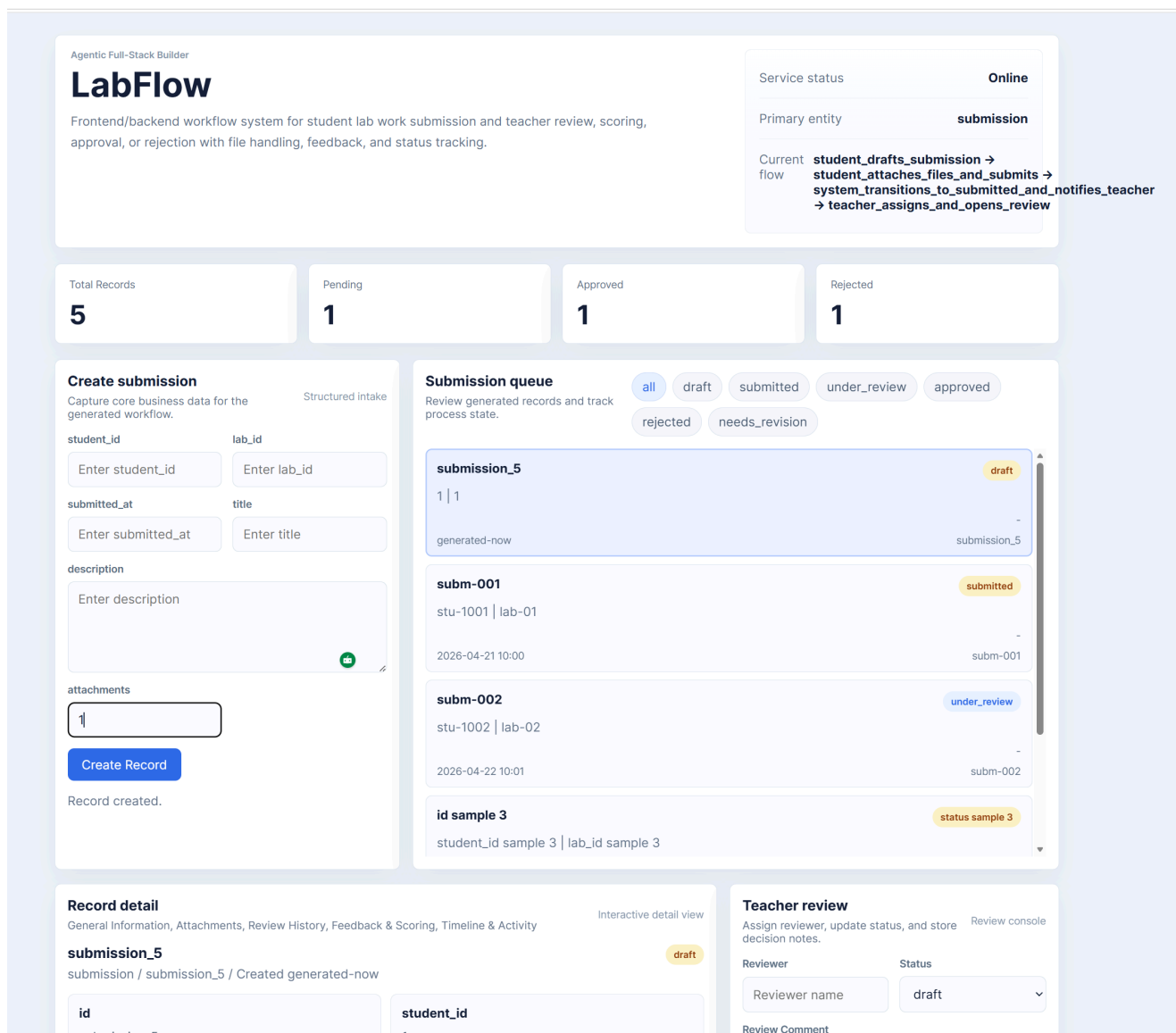
系统内部加入了模型回退机制。当某个主模型不可用时，会自动回退到后备模型，保证实验流程可以完整执行。

4.3 协同关系

本实验采用串联协作机制，而不是四个模型各自独立输出：

1. 规划智能体先执行。
2. 后端智能体读取规划结果继续设计。
3. 前端智能体同时读取规划结果与后端结果继续设计。
4. 视觉评审智能体读取前三个智能体结果，再结合结构图完成评审。

5. 系统展示



该图展示的是多智能体围绕具体需求协同完成后，最终实际生成出来的前后端系统界面，而不是单纯的结果展示页或控制台展示页。图中页面已经具备真实业务系统的基本形态，包括顶部系统概览、指标统计卡片、数据录入区域、记录列表区域以及下方的详情与批阅操作区域。

从该实例可以看出，多智能体协同的最终目标是生成一个可直接运行和观察的前后端系统。规划智能体负责给出系统目标、模块组成与任务拆解；后端智能体补充接口设计、数据模型与业务流程；前端智能体据此生成页面结构、录入表单、列表展示和交互界面；视觉评审智能体则对页面布局和展示效果进行补充分析。最终形成的不是抽象说明，而是一套能够体现真实业务流程的系统原型。

为便于同学们参考真实实现，本实验对应的 Kaggle Notebook 代码路径为：

<https://www.kaggle.com/code/richardyangwu/agents>

该链接给出了实验中多智能体生成前后端系统的完整参考实现，可结合本实验说明对照理解。

6. 实验步骤

为了保证同学们能够完整复现实验过程，本节保留从 Kaggle 生成系统到本地运行测试的完整步骤说明。

6.1 Kaggle 中的多智能体生成步骤

实验建议首先在 Kaggle Notebook 中完成多智能体生成过程。参考代码路径为：

<https://www.kaggle.com/code/richardyangwu/agents>

具体步骤如下。

1. 准备 Kaggle 工作目录

首先确认实验代码已经放入 Kaggle 工作空间，并能够访问 `build_system.py`。

```
from pathlib import Path

WORKDIR = Path("/kaggle/working")
print(WORKDIR)
```

2. 配置模型接口密钥

由于实验需要真实调用 OpenRouter 上的多个模型，因此需要先设置环境变量。

```
import os

os.environ["OPENROUTER_API_KEY"] = "你的_OPENROUTER_API_KEY"
```

3. 编写系统需求

需求应尽量明确业务对象、角色关系和交互流程。例如：

```
requirement = """
Build a frontend/backend system where students submit lab work and teachers
review,
```

```
score, approve, reject, and track the full submission workflow.
"""
```

4. 调用多智能体生成脚本

运行 `build_system.py` 后，系统会依次调用规划智能体、后端智能体、前端智能体和评审智能体，并将结果组织成可落地的工程文件。

```
!python /kaggle/working/kaggle_agentic_builder/build_system.py \
--requirement "$requirement"
```

5. 查看生成后的工程文件

生成完成后，应检查输出目录中是否已经出现后端主程序、页面模板、前端脚本、样式文件和数据文件。

```
!find /kaggle/working/generated_projects -maxdepth 2 -type f | sort
```

6.2 结合代码理解实验过程

为了帮助同学们将实验步骤与代码对应起来，下面给出几个关键代码片段，并说明其在实验中的作用。

在 `build_system.py` 中，多智能体协同生成的核心流程如下：

```
planner_resp = call_with_fallback(api_key, TEXT_MODELS["planner"],
planner_prompt(requirement))
backend_resp = call_with_fallback(api_key, TEXT_MODELS["backend"],
backend_prompt(requirement, planner_context))
frontend_resp = call_with_fallback(api_key, TEXT_MODELS["frontend"],
frontend_prompt(requirement, planner_context, backend_context))
reviewer_resp = call_with_fallback(api_key, TEXT_MODELS["reviewer"],
reviewer_prompt(requirement, planner_context, backend_context, frontend_context))
```

这段代码说明：实验并不是由单个模型一次性输出完整项目，而是先由规划智能体给出整体方案，再由后端智能体补充接口与数据结构，随后由前端智能体生成页面与交互信息，最后由评审智能体对结果进行检查与修正建议。

在同一脚本中，生成结果会被进一步整理并写入工程目录：

```
write_file(project_dir / "app.py", render_app_py())
write_file(templates_dir / "index.html", render_index_html(spec))
write_file(static_dir / "styles.css", render_styles_css())
write_file(static_dir / "app.js", render_app_js(data))
```

这段代码说明：多智能体输出的并不是停留在文本层面的描述，而是会被真正转换成后端程序、前端页面、样式和交互脚本，最终形成一个可以运行的前后端项目。

在生成出的 `app.py` 中，后端服务通过如下接口支撑页面运行：

```
@app.get("/api/dashboard")
def dashboard():
    ...

@app.post("/api/records")
def create_record():
    ...

@app.patch("/api/records/<record_id>/review")
def review_record(record_id: str):
    ...
```

这一部分说明：最终系统已经具备真实的业务处理能力，包括读取首页数据、创建记录、更新批阅结果等，而不是只有静态页面或文字说明。

6.3 将生成项目下载到本地并完成测试

在 Kaggle 中完成多智能体生成后，最后一步不是停留在网页代码展示层面，而是应将生成出的项目下载到本地，进行真实启动和交互测试。

1. 在 Kaggle 中打包生成结果

将生成目录压缩，便于下载到本地机器。

```
!cd /kaggle/working && zip -r generated_system.zip generated_projects
```

2. 下载压缩包到本地

将 `generated_system.zip` 从 Kaggle 下载到个人计算机，并解压到本地目录。

3. 进入具体生成项目目录并安装依赖

在本地命令行中进入生成出来的项目目录，执行如下命令：

```
pip install -r requirements.txt  
python app.py
```

4. 在浏览器中访问本地服务

启动成功后，可在浏览器中访问：

```
http://127.0.0.1:7860
```

5. 验证系统功能

本地测试时，应至少确认以下几个方面：

1. 页面能够正常打开；
2. 首页统计信息能够正常显示；
3. 可以创建新的业务记录；
4. 可以点击记录查看详情；
5. 可以完成批阅、状态更新和结果回写。

7. 实验要求与实践方式

本实验不限定唯一运行流程，学生可根据自身理解自由组织多智能体编程过程，但应满足以下基本要求：

1. 至少包含规划、后端、前端三个核心智能体，鼓励加入评审、测试或运维类辅助智能体。
2. 智能体之间应存在明确的上下游关系，而不是彼此孤立地产出结果。
3. 最终成果必须落到真实的前后端系统，而不是仅停留在文本方案或静态截图层面。
4. 前端页面应能够体现系统结构、接口信息、协同过程或原型效果中的至少三个方面。
5. 实验结束后，应能够结合最终结果说明各智能体分别完成了什么工作，以及它们之间如何协同。

在实践过程中，学生可以自由选择需求主题、提示词组织方式、智能体数量与调用顺序，但需要确保整个实验过程具有完整性、可解释性与可验证性。

8. 实验结果

本实验的最终结果应体现为一套由多智能体协同生成的真实前后端系统。也就是说，实验完成后，学生应能够看到实际网页界面、实际业务表单、实际记录列表以及实际处理流程，而不是仅得到若干段智能体输出文本。

结合本实验示例，生成结果已经能够体现一个完整业务系统的基础形态，包括页面总览、数据录入、列表展示、详情查看和处理操作等部分。这说明多智能体协同不仅可以完成系统方案拆解，还可以继续推动到前后端页面生成这一更接近工程实践的阶段。

需要说明的是，实验完成后应将多智能体生成得到的网页代码或工程文件下载到本地环境，实际启动前后端服务，并在浏览器中查看运行效果，不能仅停留在文本结果、截图结果或静态页面展示层面。

系统当前具备如下可验证能力：

1. 可围绕同一需求组织多个智能体协同工作。
2. 可输出前后端系统所需的结构化设计结果。
3. 可进一步生成对应的网页代码或系统原型。
4. 可在本地启动后观察真实页面效果。
5. 可围绕具体业务数据执行录入、查看和处理操作。

9. 思考题

1. 如果进一步扩展为代码自动生成系统，规划智能体还需要补充哪些结构化字段？
2. 如果引入测试智能体、运维智能体，协同顺序应如何调整？
3. 如果需要支持更复杂的前后端工程落地，哪些智能体职责还应继续细化？

10. 实验总结

本实验真实展示了多智能体编程实现前后端系统的完整过程。最终结果不是一份停留在文字层面的设计说明，而是一个可以启动、可以调用模型、可以观察交互的前后端系统。通过该实验，学生能够更直观地理解多智能体在系统设计、模块分工、前后端协作和结果验证中的作用。